DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Find Me If You Can

CMSC 197 GDD Case Study 3

Prepared by: Rene Andre B. Jocsing

Published: March 25, 2026

Case Study Objectives

- Understand why different gameplay questions require different pathfinding algorithms
- Analyze variable movement cost systems and data-driven terrain configuration
- Trace the three-phase AI decision pipeline from tactical scoring to partial movement
- Adapt the architecture to Godot using `AStarGrid2D` and custom Dijkstra flood fill

---

Background

If you have read the Component System and State System case studies, you know that Lex Talionis is an open-source engine for building turn-based strategy RPGs in the *Fire Emblem* style. The Component System handles *what things are*—items, skills, behaviors. The State System handles *what happens*—game flow, menus, combat sequences. The **Pathfinding and AI Navigation System** handles *how units move*—both for the human player (highlighting valid move tiles, drawing movement arrows) and for the AI (deciding where to go and who to attack).

This is the system that answers questions like:

- "Which tiles can this unit reach with 5 movement points?"
- "What is the shortest path from here to there, avoiding mountains and enemies?"
- "Which enemy should the AI attack, from which position, with which weapon?"
- "The AI cannot reach the target this turn—how far should it walk toward it?"

Context: Pathfinding Is Not a Single Problem

Pathfinding is one of the most-studied problems in game development. If you have taken an algorithms class, you have seen Dijkstra's algorithm and A*. But applying them to an actual game—especially a turn-based SRPG—involves problems that the textbook versions do not address.

Multiple algorithms for different questions. "What can I reach?" is a different question from "What is the fastest route to a specific tile?" Each needs a different algorithm.

Variable movement costs. A forest tile costs 2 movement points for infantry but 3 for cavalry and 1 for flying units. The same map has *different* cost graphs for different unit types.

Dynamic obstacles. Units block tiles. Allies can be walked through; enemies cannot. Invisible enemies (fog of war) do not block. This changes every turn.

Pathfinding is not AI. Finding the shortest path is only half the problem. The AI must also decide *where* to go, *who* to attack, and *which weapon* to use. Pathfinding is a tool the AI uses, not the AI itself.

Budget-constrained movement. In an SRPG, a unit does not traverse the full path in one turn—it moves as far as its movement points allow and stops. The AI needs a "walk as far as I can along this path" algorithm. Textbook A* has no built-in concept of partial traversal.

The Problem

The Wrong Algorithm Problem

Your first instinct might be to use A* for everything:

```
# O(N * A*) -- runs A* 600 times for a 30x20 map
for every_tile in map:
    path = a_star(unit.position, every_tile)
    if path_cost(path) <= unit.movement:
        valid_moves.add(every_tile)
```

This is $O(N \times A^*)$ where N is the number of tiles on the map. Dijkstra's algorithm does this in a single pass—run it once from the unit's position, and it gives you *all* reachable tiles simultaneously.

The Hardcoded Cost Problem

```
def get_cost(tile):
    if tile == "forest": return 2
    elif tile == "mountain": return 4
    elif tile == "water": return 99  # impassable
    else: return 1
```

This breaks the moment you add flying units (who ignore terrain), mounted units (who struggle in forests), or a "Water Walk" skill. You need a cost *table*—a 2D lookup of `(unit_movement_type, terrain_type) → cost`.

The Coupled AI Problem

```
def ai_think(unit):
    for enemy in enemies:
        path = a_star(unit.position, enemy.position)
        damage = compute_damage(unit, enemy)
        score = damage / len(path)
        if score > best_score:
            best = enemy
    move_along(path)
    attack(best)
```

This conflates three separate concerns: (1) where the unit can physically go, (2) what the optimal path is, and (3) what the best tactical decision is. Real AI must evaluate `(item, target, position)` triples, and the pathfinding layer should provide movement data, not make tactical decisions.

Why This Approach?

Lex Talionis solves these problems with a **layered architecture**:

1. **Three algorithms for three questions**: Dijkstra for reachability ("what can I reach?"), A* for pathing ("how do I get to X?"), Theta* for smooth movement in free-roam mode
2. **Data-driven movement costs**: A `movement_group × terrain_type → cost` lookup table (`DB.mcost`), configured in the editor. Each unit class has a movement group; each terrain has a movement type. Grids are pre-computed per movement group at level load.
3. **Facade pattern**: `PathSystem` is a high-level API that hides algorithmic details. Callers say `get_valid_moves(unit)` or `get_path(unit, position)`—they never import `Dijkstra` or `AStar` directly.
4. **AI as a consumer**: The AI controller uses `PathSystem` as a service. It asks for valid moves, asks for paths, then makes tactical decisions using a utility-scoring system. Pathfinding and AI are cleanly separated.
5. **Travel algorithm**: A dedicated `travel_algorithm()` handles partial movement—given a path and a movement budget, it returns the farthest reachable position along that path.

Architecture Deep Dive

The Grid and Node Model

Everything starts with the `Node`—a single tile in the movement grid:

```
class Node():
    __slots__ = ['reachable', 'cost', 'x', 'y', 'parent', 'g', 'h', 'f', 'true_f']

    def __init__(self, x: int, y: int, reachable: bool, cost: float):
        # Can this tile be traversed at all?
        self.reachable: bool = reachable
        # Movement points to enter this tile
        self.cost: float = cost
        self.x: int = x
        self.y: int = y
        self.reset()

    def reset(self) -> None:
        self.parent: Optional[Node] = None
        self.g: float = 0      # Actual cost from start
        self.h: float = 0      # Estimated cost to goal (heuristic)
        self.f: float = 0      # Total estimated cost (g + h)
        self.true_f: float = 0 # f without the directional nudge
```

Key Design Points

- `__slots__`: Performance optimization—no `__dict__` per node, saving memory for large grids
- `reachable` **vs** `cost`: A tile with `cost >= 99` is impassable, but `reachable` is checked *before* cost evaluation—walls are rejected in one boolean test
- `true_f` **vs** `f`: A* uses a slight directional nudge to prefer aesthetically straight paths; `true_f` stores the honest cost for limit-checking while `f` includes the nudge for tie-breaking

At level load, `GameBoard` pre-computes one grid per movement group:

```
def init_movement_grid(self, movement_group, tilemap, mtype_grid):
    grid = Grid[Node]((self.width, self.height))
```

```
    for x in range(self.width):
        for y in range(self.height):
            mtype = mtype_grid.get((x, y))
            cost = DB.mcost.get_mcost(movement_group, mtype) if mtype else 1
            grid.append(Node(x, y, cost < 99, cost))
    return grid
```

A “Flying” unit gets a grid where mountains cost 1, forests cost 1, and water costs 1. A “Foot” unit gets a grid where mountains cost 4, forests cost 2, and water costs 99. Same map; different cost landscapes.

Dijkstra: Reachability Flood Fill

When the player selects a unit, the game highlights all tiles the unit can reach. This is a **single-source reachability** problem—perfect for Dijkstra:

```
class Djikstra:
    def process(self, can_move_through, movement_left) -> Set[Pos]:
        heapq.heappush(self.open, (self.start_node.g, self.start_node))
        while self.open:
            g, node = heapq.heappop(self.open)
            # EARLY TERMINATION: heap is cost-ordered; first node that
            # exceeds the budget means all remaining nodes do too
            if g > movement_left:
                return {(n.x, n.y) for n in self.closed}
            self.closed.add(node)
            for adj in self._get_manhattan_adj_nodes(node):
                if adj.reachable and adj not in self.closed:
                    if can_move_through((adj.x, adj.y)):
                        if (adj.g, adj) in self.open:
                            if adj.g > node.g + adj.cost:
                                self._update_node(adj, node)
                                heapq.heappush(self.open, (adj.g, adj))
                        else:
                            self._update_node(adj, node)
                            heapq.heappush(self.open, (adj.g, adj))
        return {(n.x, n.y) for n in self.closed}
```

The algorithm expands outward from the start position, accumulating movement costs. Because the min-heap guarantees we always process the cheapest node first, the moment we pop a node that exceeds `movement_left`, we know every remaining node would also exceed it—early termination is sound.

Why Dijkstra and not BFS? BFS assumes uniform cost. In an SRPG, a forest costs 2 while a plain costs 1. BFS would report tiles as reachable or unreachable incorrectly.

Why not A*? A* is for finding the shortest path to a *specific* goal. We have no goal—we want *all* reachable tiles. A* with no goal degenerates into Dijkstra with the overhead of computing a useless heuristic.

A*: Optimal Single-Path Search

When the AI needs to navigate to a specific target, or when the game draws the movement arrow, it uses A*:

```python
class AStar:
    def _get_heuristic(self, node):
        """Manhattan distance + directional nudge."""
        dx1 = node.x - self.end_node.x
        dy1 = node.y - self.end_node.y
        h = abs(dx1) + abs(dy1)
        # Cross-product nudge: prefer paths along the start-to-goal line
        dx2 = self.start_node.x - self.end_node.x
        dy2 = self.start_node.y - self.end_node.y
        cross = abs(dx1 * dy2 - dx2 * dy1)
        return h + cross * .001  # 0.001 keeps paths aesthetically straight

    def process(self, can_move_through, adj_good_enough=False,
                limit=None, max_movement_limit=999):
        heapq.heappush(self.open, (self.start_node.f, self.start_node))
        while self.open:
            f, node = heapq.heappop(self.open)
            self.closed.add(node)
            if limit is not None and node.true_f > limit:
                return []  # Cost budget exceeded
            if node is self.end_node or (adj_good_enough and node in self.adj_end):
                return self._return_path(node)
            for adj in self._get_adj_nodes(node):
                if adj.reachable and adj not in self.closed:
                    if can_move_through((adj.x, adj.y)) \
                            and adj.cost <= max_movement_limit:
                        if (adj.f, adj) in self.open:
                            if adj.g > node.g + adj.cost:
                                self._update_node(adj, node)
                                heapq.heappush(self.open, (adj.f, adj))
                        else:
                            self._update_node(adj, node)
                            heapq.heappush(self.open, (adj.f, adj))
        return []
```

Key Design Points

- `adj_good_enough`: For the AI, reaching a tile *adjacent* to the target is usually sufficient (you attack from an adjacent tile). This avoids pathing to a tile occupied by the enemy.
- `limit`: The AI can cap how far it is willing to search. Uses `true_f` (without the nudge) so the limit is honest.
- `max_movement_limit`: Tiles with cost exceeding this are treated as impassable—prevents the AI from planning paths through terrain it cannot cross in one turn.
- `set_goal_pos()`: Allows re-using the same A* instance with a new goal without re-allocating the grid. `SecondaryAI` uses this to evaluate multiple targets efficiently.

Theta*: Smooth Paths for Free Roam

Lex Talionis has a free-roam mode where units move continuously rather than tile-to-tile. A* produces jagged, staircase-like paths on a grid. `ThetaStar` inherits from `AStar` and produces smoother paths by allowing shortcuts when there is line-of-sight:

```python
class ThetaStar(AStar):
    def _update_node(self, adj, node):
```

```
        # If there is line-of-sight to the PARENT of the current node,
        # skip the current node -- connect adj directly to grandparent
        if node.parent and self._line_of_sight(node.parent, adj):
            adj.g = node.parent.g + adj.cost
            adj.parent = node.parent  # Corner cut
        else:
            adj.g = node.g + adj.cost
            adj.parent = node
        adj.h = self._get_heuristic(adj)
        adj.f = adj.h + adj.g

    def _line_of_sight(self, node1, node2):
        """Bresenham line check: is there a clear line between two nodes?"""
        def blocked(pos):
            return not self.grid.get(pos).reachable
        return bresenham_line_algorithm.get_line(
            (node1.x, node1.y), (node2.x, node2.y), blocked)
```

When updating a node's parent, Theta* checks if there is line-of-sight to the *grandparent*. If so, it sets the grandparent as the direct parent, cutting the corner. The result is paths that follow straight lines through open areas rather than zigzagging along grid axes.

Movement Costs and Terrain

Movement costs are stored in a database table—a 2D lookup:

```
def get_movement_group(unit_to_move):
    """Get the unit's movement type: 'Flying', 'Mounted', 'Foot', etc."""
    movement_group = skill_system.movement_type(unit_to_move)
    if not movement_group:
        movement_group = DB.classes.get(unit_to_move.klass).movement_group
    return movement_group

def get_mcost(unit_to_move, pos):
    """How much does it cost THIS unit to enter THIS tile?"""
    terrain_nid = game.get_terrain_nid(game.tilemap, pos)
    terrain = DB.terrain.get(terrain_nid)
    movement_group = get_movement_group(unit_to_move)
    return DB.mcost.get_mcost(movement_group, terrain.mtype)
```

The data model is a grid of costs:

	Plain	Forest	Mountain	Water	Sand
Foot	1	2	4	99	2
Mounted	1	3	99	99	3
Flying	1	1	1	1	1
Armored	1	2	4	99	3

Values are set in the Lex Talionis editor. A cost of 99 or higher means “impassable”—the grid marks those nodes as `reachable = False`. Skills can override movement type at runtime: a “Water Walk” skill changes the unit’s effective movement group without touching pathfinding code.

Obstacle Handling

Static terrain is baked into the movement grid at level load. Dynamic obstacles (units) are handled by a callback:

```
def can_move_through(self, team, pos):
    """Can a unit of the given team pass through this position?"""
    unit_team = self.get_team(pos)
    if not unit_team or team in DB.teams.get_allies(unit_team):
        return True  # Empty tile or ally -- pass through
    if team == 'player' or DB.constants.value('ai_fog_of_war'):
        if not self.in_vision(pos, team):
            return True  # Enemy is fogged -- treat as empty
    return False  # Visible enemy is blocking
```

This callback is passed into pathfinding algorithms as a parameter. The pathfinding code never touches the game board directly—it calls the function it was given. The same `Dijkstra` class handles player movement (allies pass-through), AI movement (allies pass-through), and AI retreat (allies block) by swapping the callback.

The Path System Facade

Callers never instantiate `Dijkstra` or `AStar` directly. They use `PathSystem`:

```
class PathSystem():
    def get_valid_moves(self, unit, force=False, witch_warp=True) -> Set[Pos]:
        """All tiles this unit can reach this turn. Uses Dijkstra."""
        mtype = movement_funcs.get_movement_group(unit)
        grid = self.game.board.get_movement_grid(mtype)
        pathfinder = pathfinding.Djikstra(unit.position, grid)
        movement_left = unit.get_movement() if force else unit.movement_left

        if skill_system.pass_through(unit):
            can_move_through = lambda adj: True  # Ghosts ignore all obstacles
        else:
            can_move_through = partial(self.game.board.can_move_through, unit.team)

        valid_moves = pathfinder.process(can_move_through, movement_left)
        valid_moves.add(unit.position)
        if witch_warp:
            valid_moves |= set(skill_system.witch_warp(unit))
        return valid_moves

    def get_path(self, unit, position, ally_block=False,
                 use_limit=None, free_movement=False) -> List[Pos]:
        """Optimal path from unit to goal. Uses A* or Theta*."""
        mtype = movement_funcs.get_movement_group(unit)
        grid = self.game.board.get_movement_grid(mtype)

```

```
        if skill_system.pass_through(unit) or free_movement:
            can_move_through = lambda adj: True
        elif ally_block:
            can_move_through = partial(
                self.game.board.can_move_through_ally_block, unit.team)
        else:
            can_move_through = partial(self.game.board.can_move_through, unit.team)

        if free_movement:
            pathfinder = pathfinding.ThetaStar(unit.position, position, grid)
        else:
            pathfinder = pathfinding.AStar(unit.position, position, grid)
        return pathfinder.process(can_move_through, limit=use_limit)
```

This is the **Facade pattern**: a simple interface over complex subsystems. The caller says `game.path_system.get_valid_moves(unit)` and does not care whether Dijkstra, A*, or some future algorithm runs underneath.

AI Integration

The AI Decision Pipeline

The AI does not just pathfind—it makes *decisions*. Lex Talionis uses a **three-phase pipeline**:

```
Phase 1: THINK
  - Try PrimaryAI (can I attack someone reachable right now?)
      For each (item, target, position): score utility
      Pick the triple with the maximum score
  - If no attack found: Try SecondaryAI (navigate toward best target)
      For each potential target: A* pathfind, score by distance + damage
      travel_algorithm -> walk as far as movement budget allows

Phase 2: MOVE
  - Execute movement along chosen path via state machine + animation

Phase 3: ATTACK (if PrimaryAI found a target)
  - Execute combat

Phase 4: CANTO (post-attack movement, if unit has Canto skill)
  - canto_retreat: move away from concentrated enemies
```

The AI controller has a simple state machine: `Init → Primary → Secondary → Done`. It interleaves thinking across frames—each call to `think()` processes for up to half a frame (`FRAMERATE/2` milliseconds), then yields. This prevents the AI from freezing the game.

```
def think(self):
    time = engine.get_time()
    while True:
        over_time = engine.get_true_time() - time >= FRAMERATE/2

        if self.state == 'Init':
            self.set_next_behaviour()
            if self.behaviour.action == "Attack":
                self.inner_ai = self.build_primary()
                self.state = "Primary"
```

```
            elif self.behaviour.action == "Move_to":
                self.inner_ai = self.build_secondary()
                self.state = "Secondary"

        elif self.state == 'Primary':
            done, target, position, item = self.inner_ai.run()
            if done:
                if target:
                    self.state = "Done"
                else:
                    self.inner_ai = self.build_secondary()
                    self.state = "Secondary"  # Fall back to navigation

        elif self.state == 'Secondary':
            done, position = self.inner_ai.run()
            if done:
                self.state = "Done"

        if self.state == 'Done':
            return True
        if over_time:
            break  # Yield to game loop
    return False
```

PrimaryAI: Close-Range Combat Targeting

`PrimaryAI` answers: "Given the tiles I can reach *right now*, what is the best `(item, target, position)` combination?" It iterates over a three-level loop:

```
For each usable item:
    For each valid target in item range from any valid move:
        For each position I can strike from:
            Compute utility score
            Track best (item, target, position) triple
```

The AI *temporarily teleports* the unit to each candidate position to evaluate combat stats (which can depend on terrain bonuses, adjacency, etc.), then teleports back. This avoids the complexity of full what-if simulation:

```
def run(self):
    target = self.valid_targets[self.target_index]
    item = self.items[self.item_index]
    move = self.possible_moves[self.move_index]

    if self.unit.position != move:
        self.quick_move(move)  # Temporarily teleport

    if DB.constants.value('line_of_sight'):
        valid = line_of_sight.line_of_sight([move], [target], max_range)
        if not valid:
            return  # Cannot see target from here

    self.determine_utility(move, target, item)
```

SecondaryAI: Long-Range Navigation

When `PrimaryAI` finds no viable attack, `SecondaryAI` takes over: "I cannot attack anyone this turn—who should I walk *toward*?"

```
class SecondaryAI():
    def __init__(self, unit, behaviour):
        self.unit = unit
        self.all_targets = get_targets(unit, behaviour)
        movement_group = movement_funcs.get_movement_group(unit)
        self.grid = game.board.get_movement_grid(movement_group)
        # Pre-build the A* pathfinder -- reused across all target evaluations
        self.pathfinder = pathfinding.AStar(unit.position, None, self.grid)

    def run(self):
        if self.available_targets:
            target = self.available_targets.pop()
            path = self.get_path(target)
            if path:
                tp = self.compute_priority(target, len(path))
                if tp is not None and (self.max_tp is None or tp > self.max_tp):
                    self.max_tp = tp
                    self.best_target = target
                    self.best_path = path
        elif self.best_target:
            self.best_position = game.path_system.travel_algorithm(
                self.best_path, self.unit.movement_left,
                self.unit, self.grid)
            return True, self.best_position
        else:
            return True, None
        return False, None

    def get_path(self, goal_pos):
        self.pathfinder.set_goal_pos(goal_pos)  # Reuse A* instance, new goal
        adj_good_enough = self.behaviour.target not in ('Event', 'Terrain')
        can_move_through = partial(game.board.can_move_through, self.unit.team)
        path = self.pathfinder.process(
            can_move_through,
            adj_good_enough=adj_good_enough,
            limit=self.get_limit(),
            max_movement_limit=self.unit.get_movement()
        )
        self.pathfinder.reset()
        return path
```

`SecondaryAI` *reuses* a single `AStar` instance across all targets—it calls `set_goal_pos()` to retarget and `reset()` to clear the visited set. This avoids repeated grid allocation and is a significant performance optimization when evaluating dozens of potential targets.

Utility Scoring

The AI does not simply pick the closest enemy—it scores each option using a **weighted utility function**:

```
# PrimaryAI: close-range scoring
def default_priority(self, main_target, item, move):
```

```
    raw_damage = combat_calcs.compute_damage(self.unit, main_target, item, ...)
    lethality = clamp(raw_damage / main_target.get_hp(), 0, 1)
    accuracy  = clamp(combat_calcs.compute_hit(...) / 100, 0, 1)
    # Large bonus for guaranteed kills
    offense_term = 3 if lethality * accuracy >= 1 \
                   else lethality * accuracy * num_attacks

    counter_damage = combat_calcs.compute_damage(main_target, self.unit, ...)
    defense_term   = 1 - counter_damage * counter_accuracy * (1 - first_strike)

    # Tiebreaker: prefer not moving too far
    distance_term  = (max_dist - distance(move, orig_pos)) / max_dist

    offense_weight = offense_bias / (offense_bias + 1)
    defense_weight = 1 - offense_weight
    return process_terms([
        (offense_term, offense_weight),
        (defense_term, defense_weight),
        (distance_term, .0001),  # Only breaks ties
    ])
```

```
# SecondaryAI: long-range scoring
def compute_priority(self, target, distance):
    distance_term = 1 - math.log(distance) / 4  # Logarithmic decay
    terms = [(distance_term, 60)]

    if self.behaviour.action == "Attack" and enemy:
        terms += self.default_priority(enemy)
    elif self.behaviour.action == "Support" and ally:
        help_term = (ally.get_max_hp() - ally.get_hp()) / ally.get_max_hp()
        terms.append((help_term, 100))  # Prioritize injured allies

    return utils.process_terms(terms)
```

The `offense_bias` parameter is configurable per AI profile in the editor, letting designers create aggressive AI (high bias), defensive AI (low bias), or balanced AI—without touching code.

The Travel Algorithm

When the AI's target is out of reach, it must walk *as far as possible* along the planned path:

```
def travel_algorithm(self, path, moves, unit, grid) -> Pos:
    """
    Given a path (goal-first, start-last) and a movement budget,
    returns the farthest position the unit can reach along the path.
    """
    if not path:
        return unit.position

    moves_left = moves
    through_path = 0
    for position in path[::-1][1:]:  # Walk from start toward goal
        moves_left -= grid.get(position).cost
        if moves_left >= 0:
            through_path += 1
        else:
```

```
            break

    # Don't step on another unit
    while through_path > 0 and any(
        other.position == path[-(through_path + 1)]
        for other in self.game.units if unit is not other
    ):
        through_path -= 1

    return path[-(through_path + 1)]
```

This reverses the path (stored goal-first), walks forward accumulating costs, stops when the budget runs out, then backtracks if another unit occupies the target tile. The result is $O(\text{path length})$—simple, correct, and fast.

Supporting Systems

Line of Sight

When the `line_of_sight` database constant is enabled, units cannot attack through walls. The system uses **Bresenham's line algorithm** to check whether a clear line exists between two positions:

```
def get_line(start, end, get_opacity) -> bool:
    """
    Trace a rasterized line from start to end.
    Returns True if no opaque tile blocks the line.
    """
    # ... Bresenham's algorithm with diagonal handling ...
    # For each tile the line passes through:
    #   if get_opacity(tile) is True: line is blocked
    return True  # Clear line of sight
```

The `get_opacity` callback makes the algorithm reusable: Theta* uses it for path smoothing (checking walkability); the combat system uses it for attack visibility (checking opacity). Same algorithm, two different meanings of "blocked."

Fog of War

The fog of war system maintains a per-team visibility grid updated when units move:

```
def update_fow(self, pos, unit, sight_range):
    grid = self.fog_of_war_grids[unit.team]
    for cell in grid.cells():
        cell.discard(unit.nid)  # Clear old vision
    if pos:
        positions = game.target_system.find_manhattan_spheres(
            range(sight_range + 1), *pos)
        for position in positions:
            grid.get(position).add(unit.nid)  # Add new vision
```

Fog of war interacts with pathfinding through the `can_move_through` callback: if the AI cannot see a tile (it is fogged), it assumes it can move through it—even if an enemy is there. This creates realistic AI behavior: units move toward fogged areas without cheating by seeing hidden enemies.

Applying to Other Game Genres

The principles in this system transfer broadly across game types. The table below summarizes what changes and what does not.

Genre	What transfers directly	What changes
Open-world / Action RPG	Data-driven cost tables; facade pattern; same Dijkstra/A* algorithms on a navmesh	8-directional or continuous movement; local avoidance (RVO/steering) on top of global pathfinding; hierarchical pathfinding for large maps
Real-Time Strategy	Dijkstra for reachability; cost tables for terrain variation; AI pipeline separation	Flow fields replace per-unit A* at scale; steering and flocking for physical unit size; time-sliced computation across many units simultaneously
Roguelike / Dungeon Crawler	Grid-based pathfinding with variable costs; dynamic obstacle callbacks; utility scoring for AI	Procedural maps require incremental grid construction; many entities with simpler AI (no complex PrimaryAI needed)
Platformer	Navigation graph with nodes as safe positions and edges as actions; passable-callback for one-way platforms	Gravity and physics make movement arc-based, not free; A* edges encode jump sequences, not directions; action sequences replace tile paths

Universal Principles

Regardless of genre, these principles apply:

1. **Use the right algorithm for the question**: Do not run A* when you need Dijkstra. "What can I reach?" and "How do I get to X?" are different problems.
2. **Data-driven costs**: Use a lookup table designers can tune. The `(entity_type, terrain_type) → cost` pattern scales to any game.
3. **Separate pathfinding from AI**: Pathfinding answers "how do I get there?" AI answers "should I go there?" Keep them in separate layers.
4. **Budget your computation**: Time-slice across frames, cache pre-computed grids, and limit search radius with the A* `limit` parameter.
5. **Callbacks for dynamic state**: Pass obstacle-checking as a function parameter, not a hardcoded check. The same algorithm then handles allies, enemies, fog of war, and pass-through skills.
6. **Partial traversal is a first-class concept**: "Walk as far along this path as my budget allows" is an essential game primitive. Textbook pathfinding stops at "here is the path."

Adapting to Godot

Godot 4 provides `AStarGrid2D` and `NavigationServer2D` as building blocks, but you still need custom Dijkstra for reachability, custom AI logic for decisions, and the data-driven cost table for terrain variation.

Step 1: Grid Pathfinding with AStarGrid2D

```gdscript
class_name GridPathfinder extends Node

var grid: AStarGrid2D

func setup(tilemap: TileMap, tile_size: Vector2i) -> void:
    grid = AStarGrid2D.new()
    grid.region = tilemap.get_used_rect()
    grid.cell_size = Vector2(tile_size)
    grid.diagonal_mode = AStarGrid2D.DIAGONAL_MODE_NEVER  # 4-directional
    grid.update()
    for cell in tilemap.get_used_cells(0):
        var terrain_id: TileData = tilemap.get_cell_tile_data(0, cell)
        var cost: float = _get_terrain_cost(terrain_id)
        if cost >= 99:
            grid.set_point_solid(cell, true)  # Impassable
        else:
            grid.set_point_weight_scale(cell, cost)

func get_path_to(from_pos: Vector2i, to_pos: Vector2i) -> PackedVector2Array:
    return grid.get_point_path(from_pos, to_pos)
```

Step 2: Custom Dijkstra for Reachability

Godot's `AStarGrid2D` has no flood-fill method, so this must be custom:

```gdscript
class_name DijkstraFlood extends RefCounted

func find_reachable(start: Vector2i, movement: float,
                    cost_func: Callable,
                    passable_func: Callable) -> Array[Vector2i]:
    var open_heap: Array = [[0.0, start]]
    var costs: Dictionary = {start: 0.0}
    var closed: Dictionary = {}

    while not open_heap.is_empty():
        open_heap.sort_custom(func(a, b): return a[0] < b[0])
        var current: Array = open_heap.pop_front()
        var g: float = current[0]
        var pos: Vector2i = current[1]

        if g > movement:
            break  # Early termination
        if pos in closed:
            continue
        closed[pos] = true

        for neighbor: Vector2i in _get_neighbors(pos):
            if neighbor in closed or not passable_func.call(neighbor):
                continue
            var new_g: float = g + cost_func.call(neighbor)
            if neighbor not in costs or new_g < costs[neighbor]:
                costs[neighbor] = new_g
                open_heap.append([new_g, neighbor])

    return closed.keys()

```

```
func _get_neighbors(pos: Vector2i) -> Array[Vector2i]:
    return [pos + Vector2i.UP, pos + Vector2i.DOWN,
            pos + Vector2i.LEFT, pos + Vector2i.RIGHT]
```

Step 3: Data-Driven Movement Costs

```
class_name MovementCostTable extends Resource

@export var costs: Dictionary = {
    "Foot":    {"Plain": 1, "Forest": 2, "Mountain": 4,  "Water": 99},
    "Mounted": {"Plain": 1, "Forest": 3, "Mountain": 99, "Water": 99},
    "Flying":  {"Plain": 1, "Forest": 1, "Mountain": 1,  "Water": 1},
}

func get_cost(movement_group: String, terrain: String) -> float:
    if movement_group in costs and terrain in costs[movement_group]:
        return costs[movement_group][terrain]
    return 1.0
```

Step 4: AI Controller with Pathfinding

```
class_name AIController extends Node

var pathfinder: GridPathfinder
var dijkstra: DijkstraFlood
var cost_table: MovementCostTable

func think(unit: GameUnit) -> Dictionary:
    # Phase 1: Can we attack from somewhere we can reach?
    var valid_moves: Array[Vector2i] = dijkstra.find_reachable(
        unit.grid_pos, unit.movement,
        func(pos): return cost_table.get_cost(unit.movement_group,
                                              _get_terrain(pos)),
        func(pos): return _can_move_through(unit, pos)
    )
    var best: Dictionary = _find_best_attack(unit, valid_moves)
    if not best.is_empty():
        return best

    # Phase 2: Navigate toward best reachable target
    var best_path: PackedVector2Array = []
    var best_score: float = -INF
    for target: Vector2i in _get_enemy_positions(unit):
        var path: PackedVector2Array = pathfinder.get_path_to(
            unit.grid_pos, target)
        if path.is_empty():
            continue
        var score: float = _score_target(unit, target, path.size())
        if score > best_score:
            best_score = score
            best_path = path

    if not best_path.is_empty():
        var move_to: Vector2i = _travel_along_path(
```

```
            best_path, unit.movement, unit)
        return {"action": "move", "position": move_to}

    return {"action": "wait"}

func _travel_along_path(path: PackedVector2Array, budget: float,
                        unit: GameUnit) -> Vector2i:
    var spent: float = 0.0
    var last_valid: Vector2i = Vector2i(path[0])
    for i: int in range(1, path.size()):
        var pos: Vector2i = Vector2i(path[i])
        spent += cost_table.get_cost(unit.movement_group, _get_terrain(pos))
        if spent > budget:
            break
        if not _is_occupied(pos, unit):
            last_valid = pos
    return last_valid
```

Architecture Comparison

Lex Talionis (Python)	Godot 4 (GDScript)
`Node` (pathfinding grid cell)	`AStarGrid2D` (built-in) or custom `Node`
`Dijkstra` (reachability)	Custom `DijkstraFlood` (no built-in equivalent)
`AStar` (optimal path)	`AStarGrid2D.get_point_path()`
`ThetaStar` (smooth paths)	`NavigationServer2D` (navmesh-based)
`PathSystem` (facade)	Custom `PathSystem` wrapping the above
`DB.mcost` (cost table)	`MovementCostTable` resource
`GameBoard.can_move_through`	`Callable` passed into pathfinder
`AIController` (decision pipeline)	Custom `AIController` node
`PrimaryAI` (close-range scoring)	Same pattern, GDScript implementation
`SecondaryAI` (long-range nav)	Same pattern, GDScript implementation
`travel_algorithm` (partial paths)	`_travel_along_path()` method
`bresenham_line_algorithm`	`Geometry2D` or custom Bresenham
Fog of war grid	Custom visibility grid or `TileMap` layer

Conclusion

The Lex Talionis Pathfinding and AI Navigation System demonstrates how a clean layered architecture handles the surprisingly complex problem of “how should game entities move.”

Key Takeaways

1. **Different questions need different algorithms**: Dijkstra for “what can I reach?”, A* for “how do I get to X?”, Theta* for “make the path look natural.” Do not force one algorithm to answer all three questions.
2. **Data-driven costs beat hardcoded logic**: A `(movement_group, terrain_type) → cost` table, editable by designers, handles flying units, mounted units, water-walking skills, and every other movement variation—without touching pathfinding code.

3. **Separate pathfinding from AI**: Pathfinding is a *service* that answers "can I get there and how?" The AI is a *consumer* that asks "should I go there?" Keep them in separate classes with clean interfaces.
4. **The Facade pattern hides complexity**: Callers say `get_valid_moves(unit)` and `get_path(unit, goal)`. They do not care which algorithm runs, how the cost table is structured, or how obstacle checking works.
5. **Budget computation across frames**: AI thinking is time-sliced—process for half a frame, then yield. This prevents the game from freezing during complex evaluations with many units and targets.
6. **Callbacks decouple dynamic state**: `can_move_through` is a function passed *into* the pathfinder, not a hardcoded check *inside* it. The same algorithm works for allies, enemies, fog of war, and pass-through skills.
7. **Partial traversal is a first-class concept**: The travel algorithm—"walk as far along this path as my budget allows"—is essential for multi-turn AI planning. Textbook pathfinding stops at "here is the path." Game pathfinding needs "here is how far along it I can get."

This document references our private dev fork of the Lex Talionis engine here, where I work together with some other contributors to produce a fork of the engine suitable for deployment on Steam and other commercial platforms. All code snippets are drawn from the codebase and lightly adapted for clarity. You may view the master branch of the Lex Talionis engine at my personal mirror or at the source.